

1. Ambiente operativo

L'analisi del codice sorgente e la valutazione del malware sono state eseguite all'interno di una macchina virtuale con sistema operativo Windows 10, ospitata nell'ambiente di virtualizzazione VirtualBox. La connettività di rete dell'ambiente è completamente isolata e controllata tramite una macchina virtuale pfSense, la quale gestisce l'instradamento e il monitoraggio del traffico di rete virtuale.

2. Obiettivi

Le finalità delle attività del presente report è l'analisi statica del codice sorgente della versione A del malware Mydoom, integrando l'analisi forense con un'osservazione di threat intelligence. Gli obiettivi specifici dell'intervento comprendono l'analisi forense e fecomposizione tecnica, ovvero la dissezione della struttura e la logica di funzionamento dei singoli moduli del malware, e delle le tecniche di evasione dei controlli di sicurezza, i meccanismi di persistenza e la gestione del canale di controllo remoto tramite il proxy SOCKS4

3. Panoramica del malware

Mydoom (noto anche come Novarg o Mimail.R) è uno dei worm e-mail a più rapida diffusione nella storia della cybersecurity, comparso per la prima volta nel gennaio 2004 sui sistemi operativi Microsoft Windows. La struttura del malware si basa sulla combinazione di ingegneria sociale tramite campagne di mass-mailing con allegati ingannevoli, propagazione secondaria attraverso la cartella condivisa del client *Peer-to-Peer Kaza*, installazione di una backdoor remota via *proxy SOCKS4* su porta TCP/IP 3127 ed esecuzione temporizzata di attacchi *Distributed Denial of Service* (DDoS) coordinati.

Nel corso del tempo il malware si è evoluto attraverso diverse varianti, tra cui le versioni A, B, C, D, E, U, V, W, X e AO:

Codename della versione	Funzionalità
A	La versione originaria focalizzata sull'estrazione di indirizzi e-mail dai file locali, all', invio massivo tramite motore SMTP interno, e all'apertura della backdoor di sistema TCP/IP 3127, e all'esecuzione di un attacco DDoS mirato al dominio di <i>SCO Group</i> .
B	Con questa versione risulta essere implementata l'invocazione di un attacco DDoS addizionale rivolto ai domini riferibili all'azienda <i>Microsoft</i> , e risulta essere implementato il blocco delle connessioni verso oltre 60 siti web di sicurezza e antivirus per impedire il download di strumenti di rimozione.
C, o Doomjuice	Con questa versione risulta essere implementato lo sfruttamento delle porte TCP/IP aperte dalle esecuzioni precedenti delle versioni A e B al fine di diffusione nella rete senza il vincolo di l'apertura di allegati e-mail da parte dell'utente vittima.
C e E	Queste versioni risultano intermedie e risultano possedere <i>perfezionamenti</i> sulle tecniche di diffusione del vettore, l'ampliamento dei modelli di messaggi ingannevoli, l'impiego di estensioni doppie o archivi ZIP per aggiramento di filtri <i>antispam</i> aziendali.
U, V, W e X	Queste versioni, apparse nella seconda metà del 2004, risultano introdurre nell'esecuzione del <i>malware</i> tecniche di <i>querying</i> automatico sui motori di ricerca (come Google, Yahoo! e AltaVista) al fine di raccogliere nuovi elenchi di e-mail, provocando rallentamenti su scala globale alla rete Internet.
AO	Questa versione, emersa all'inizio del 2005, risulta introdurre nell'esecuzione del <i>malware</i> consolidamenti della persistenza nei sistemi infetti. Inoltre risulta che l'eseguibile presenti un codice offuscato, e che presenti funzionalità di inserimento del controllo malevolo della macchina ospite all'interno di una botnet dedicata all'instradamento di traffico non autorizzato e allo spam.

4. Moduli del codice del *malware*

4.1. Modulo principale (`main.c`)

Il modulo `main.c` rappresenta il punto d'ingresso e l'orchestratore principale del software malevolo, gestendone l'intero ciclo di vita dall'inizializzazione dell'ambiente alla diffusione di rete. Il codice coordina il controllo dell'esecuzione unica tramite mutex, la verifica e l'impostazione della prima esecuzione e della persistenza nel registro di sistema di Windows, la copia dell'eseguibile nelle directory di sistema o temporanee, la decifratura e il caricamento in memoria di un modulo proxy DLL (`shimgapi.dll`), la generazione di un'esca visiva con Notepad,

la gestione di scadenze temporali per attacchi Denial of Service (DoS) e l'attivazione dei moduli di propagazione via P2P, e-mail di massa e scansione di rete.

L'esecuzione del programma prende avvio nella funzione `WinMain`, la quale prepara il contesto operativo inizializzando il generatore di numeri casuali con `xrand_init`, avviando il sottosistema di rete mediante la funzione `wsa_init` (che richiama l'API `WSAStartup`) e impostando le strutture temporali per la disattivazione del malware e per l'attacco DoS all'interno della struttura `sync_t`, prima di inoltrare l'esecuzione a `sync_main` e terminare con `ExitProcess`. La funzione principale di orchestrazione, `sync_main`, ottiene il tempo di avvio con `GetTickCount`, determina lo stato del primo avvio chiamando `sync_check_frun`, verifica la presenza di altre istanze attive con `sync_mutex` e genera un thread per la visualizzazione dell'esca visiva `sync_visual_th` tramite `CreateThread` se si tratta della prima esecuzione. Successivamente, `sync_main` installa e carica la DLL proxy tramite `payload_xproxy`, controlla il superamento della data limite con `sync_gettime`, garantisce l'installazione e l'avvio automatico con `sync_install` e `sync_startup`, tenta l'esecuzione dell'attacco DoS con `payload_sco`, avvia la componente P2P tramite `p2p_spread`, inizializza ed esegue il thread di invio e-mail tramite `massmail_init` e `CreateThread` diretto a `massmail_main_th`, e infine avvia il ciclo di scansione permanente della rete richiamando `scan_init`, `scan_main` e pause temporizzate con `Sleep`.

La verifica della prima installazione è eseguita dalla funzione `sync_check_frun`, che decifra il percorso della chiave di registro usando `rot13` e ne controlla l'esistenza mediante `RegOpenKeyEx` sui rami HKLM e HKCU, chiudendo l'handle con `RegCloseKey` e creando la chiave con `RegCreateKeyEx` qualora l'applicazione venga eseguita per la prima volta. La funzione `sync_mutex` gestisce l'unicità dell'istanza decodificando il nome del mutex con `rot13`, creando l'oggetto mediante `CreateMutex` e verificando la presenza di duplicati con `GetLastError`. L'installazione dell'eseguibile malevolo nel sistema è gestita da `sync_install`, che ricava il percorso del proprio processo con `GetModuleFileName`, deoffusca il nome del file di destinazione con `rot13`, ne identifica la cartella target con `GetSystemDirectory` o `GetTempPath`, ne modifica gli attributi mediante `SetFileAttributes` ed effettua la copia del file avvalendosi delle funzioni `CreateFile`, `CloseHandle`, `DeleteFile` e `CopyFile`. L'impostazione dell'esecuzione automatica è assegnata a `sync_startup`, che deoffusca percorso e nome del valore di registro tramite `rot13`, apre le chiavi di avvio con `RegOpenKeyEx`, scrive il percorso dell'eseguibile con `RegSetValueEx` e chiude la chiave con `RegCloseKey`.

L'estrazione e l'esecuzione del payload proxy sono gestite dalla funzione `payload_xproxy`, che decodifica il nome della DLL target via `rot13`, individua le directory di sistema o temporanea tramite `GetSystemDirectory` e `GetTempPath`, crea il file DLL con `CreateFile`, vi scrive il codice binario decifrato chiamando `decrypt1_to_file` e ne chiude l'handle con `CloseHandle`, per poi caricare la DLL nello spazio di memoria del processo mediante `LoadLibrary`. La funzione ausiliaria `decrypt1_to_file` applica una chiave XOR dinamica al buffer sorgente e scrive i dati decifrati su file tramite `WriteFile`. La valutazione delle condizioni di scadenza del malware è effettuata dalla funzione `sync_gettime`, la quale confronta l'orario di sistema ottenuto con `GetSystemTimeAsFileTime` rispetto alla data di disattivazione convertita tramite `SystemTimeToFileTime`.

L'attacco Denial of Service verso il target specifico è gestito da `payload_sco`, che controlla l'orario corrente tramite `GetSystemTimeAsFileTime` e `SystemTimeToFileTime` e, al raggiungimento della data prefissata, entra in un ciclo infinito che invoca ripetutamente la routine di attacco `scodos_main` intervallandola con pause `Sleep`. Infine, la funzione `sync_visual_th` esegue una tattica di diversione in un thread separato: ottiene il percorso temporaneo tramite `GetTempPath`, crea un file di testo fittizio mediante `CreateFile`, genera un buffer contenente dati casuali e ritorni a capo richiamando `xrand16` e lo scrive nel file con `WriteFile`, chiude l'handle con `CloseHandle`, avvia l'applicazione Notepad per mostrare il file con `CreateProcess`, ne attende la chiusura con `WaitForSingleObject`, rimuove il file temporaneo dal disco con `DeleteFile` e termina il thread chiamando `ExitThread`.

4.2. Moduli `lib.c` e `lib.h`

Il modulo rappresentato dai file `lib.h` e `lib.c` funge da libreria di supporto generale contenente funzioni di utilità per la manipolazione di stringhe, la decifratura e l'offuscamento tramite cifrario ROT13, il calcolo di numeri pseudo-casuali, la formattazione di date conformi allo standard SMTP, l'esecuzione di processi in modalità nascosta, la decodifica di entità HTML o URL e la verifica dinamica della connettività di rete.

La funzione `rot13c` esegue la trasposizione cifrata di un singolo carattere nell'alfabeto maiuscolo o minuscolo avvalendosi della funzione di ricerca `xstrchr`. La funzione `rot13` itera sulla stringa d'ingresso invocando ripetutamente `rot13c` per decifrare o cifrare l'intera sequenza di testo. La formattazione della data è affidata alla funzione `mk_smtpdate`, che rileva l'orario locale del sistema o converte la struttura di tempo di file tramite `GetLocalTime`, `FileTimeToLocalFileTime` e `FileTimeToSystemTime`, rileva le informazioni sul fuso orario e l'ora legale con `GetTimeZoneInformation` e compone la stringa dell'intestazione SMTP invocando `wsprintf`.

Per la generazione di numeri casuali, la funzione `xrand_init` acquisisce il tempo trascorso dall'avvio del sistema tramite `GetTickCount` per inizializzare il seme; le chiamate a `xrand16` aggiornano il seme tramite calcoli aritmetici per restituire un valore a 16 bit, mentre la funzione `xrand32` effettua due chiamate a `xrand16` per combinare i risultati e restituire un valore a 32 bit. L'esecuzione occultata dei comandi di sistema viene gestita dalla funzione `xsystem`, la quale azzerla la struttura d'avvio con `ZeroMemory`, configura la finestra come nascosta ed

esegue il processo mediante `CreateProcess`; qualora sia richiesta l'attesa del completamento del processo, `xsysem` ne sospende l'esecuzione con `WaitForSingleObject` e chiude successivamente le risorse richiamando `CloseHandle`.

La verifica dello stato di rete è implementata nella funzione `is_online`, che decodifica i nomi della DLL e dell'API di sistema tramite `rot13`, carica la libreria in memoria con `GetModuleHandle` o `LoadLibrary`, estrae l'indirizzo della procedura tramite `GetProcAddress` e invoca l'API `InternetGetConnectedState` per determinare se il sistema sia connesso a Internet. Per la gestione del testo e dell'HTML, `xmemcmpi` esegue confronti di memoria convertendo i caratteri in minuscolo, mentre `html_replace` individua i codici numerici entità HTML e li converte in testo standard (utilizzando `WideCharToMultiByte` per caratteri estesi) prima di riorganizzare i dati nel buffer mediante `memcpy` e `lstrlen`. La funzione `html_replace2` decodifica le sequenze percent-encoding degli indirizzi URL interpretando i valori esadecimali e sovrascrivendo i dati con `memcpy` e `lstrlen`. Infine, la funzione `cat_wsprintf` misura la lunghezza della stringa esistente tramite `lstrlen` e vi concatena il nuovo testo formattato richiamando l'API `wvsprintf`.

4.3. Moduli *proxy* `xproxy.c` e `client.c`

I moduli `client.c` e `xproxy.c` compongono una struttura client-server per il controllo remoto e la funzionalità di proxying SOCKS4 con capacità di esecuzione remota di payload. Il file `client.c` implementa un'utilità da riga di comando per la connessione verso una macchina remota al fine di trasmettere un file eseguibile previa autenticazione basata su un valore magico. Il file `xproxy.c` costituisce una libreria dinamica (DLL) che avvia un server proxy SOCKS4 multithread, gestisce la persistenza nel registro di Windows e la registrazione come servizio, intercetta comandi speciali per la ricezione e l'esecuzione nascosta di payload ed effettua l'instradamento del traffico di rete tra client e destinazioni remote.

Nel modulo `client.c`, l'unica funzione presente è `main`, la quale inizializza la libreria di rete mediante `WSAStartup`, apre in modalità binaria il file specificato negli argomenti con `fopen`, risolve l'hostname remoto tramite `gethostbyname`, crea un socket TCP con `socket` e stabilisce la connessione con `connect`. Successivamente, `main` invia l'instestazione di autenticazione contenente il valore numerico speciale con `send`, controlla lo stato del socket usando `select` e `recv`, legge i blocchi di dati del file con `fread` e li trasmette sulla rete tramite `send` fino al completamento, chiudendo infine la connessione con `closesocket`.

Nel modulo `xproxy.c`, la funzione d'ingresso `DllMain` intercetta l'aggancio del processo e crea un thread principale invocando `CreateThread` per eseguire `xproxy_th`. La funzione `xproxy_th` coordina la fase iniziale chiamando `regsvc9x` per nascondere il processo, `wsainit` per avviare Winsock tramite `WSAStartup`, `shellsvc_attach` per garantire la persistenza ed esegue un ciclo su un intervallo di porte di rete chiamando `socks4_main` con pause temporizzate tramite `Sleep`. La funzione `regsvc9x` decifra il nome dell'API di sistema tramite `rot13`, ottiene l'handle del modulo di sistema con `GetModuleHandle` o `LoadLibrary` ed estrae l'indirizzo della funzione con `GetProcAddress` per registrarla. La funzione `shellsvc_attach` recupera il percorso del proprio modulo tramite `VirtualQuery` e `GetModuleFileName`, decodifica la chiave di registro `InprocServer32` con `rot13` e invoca `reg_setval` per la registrazione, dove `reg_setval` crea o apre la chiave con `RegOpenKeyEx` o `RegCreateKeyEx`, imposta il valore con `RegSetValueEx` e chiude la chiave con `RegCloseKey`.

La gestione del server proxy viene avviata da `socks4_main`, la quale crea il socket di ascolto con `socket`, ne associa la porta con `bind`, lo pone in ascolto mediante `listen`, genera i thread di servizio invocando `CreateThread` verso la routine `socks4_server_th` e controlla periodicamente il numero di thread attivi richiamando `shellsvc_attach` e `GetTickCount`. La funzione `socks4_server_th` accetta le connessioni in ingresso tramite `accept`, delega la gestione del client a `socks4_client` e regola la temporizzazione del thread con `GetTickCount` e `Sleep`. La funzione `socks4_client` analizza il primo byte in ingresso con `recv` (usando la modalità `MSG_PEEK`): se il byte corrisponde al comando speciale invoca `socks4_exec`, altrimenti elabora l'instestazione SOCKS4 leggendo i dati con `recv_bytes` e `skip_until`. Qualora sia richiesta l'estensione SOCKS4A, `socks4_client` invoca `parse_socks4a` per risolvere il dominio remoto tramite `inet_addr` o `gethostbyname`, crea il socket di inoltramento con `socket`, si connette al server di destinazione con `connect`, risponde al client con `send` e avvia il relé bidirezionale richiamando `relay_socks`.

La funzione `relay_socks` gestisce lo scambio dati continuo tra client e destinazione mediante un ciclo con `select`, `recv` e `send`. Qualora venga ricevuto il comando di esecuzione, la funzione `socks4_exec` verifica il valore di controllo numerico convertito con `ntohl`, ottiene i percorsi temporanei di sistema con `GetTempPath` e `GetTempFileName`, crea un file temporaneo sul disco con `CreateFile`, ne scrive i dati ricevuti con `WriteFile`, chiude il file con `CloseHandle`, avvia l'eseguibile in modalità nascosta con `CreateProcess`, attende il termine dell'esecuzione con `WaitForSingleObject`, libera le risorse di processo con `CloseHandle`, rimuove il file con `DeleteFile` e chiude il socket con `closesocket`. Le funzioni ausiliarie `recv_bytes` e `skip_until` gestiscono rispettivamente la lettura completa dei buffer e la scansione dei caratteri terminatori invocando `recv`, mentre `rot13` e `rot13c` eseguono la decodifica dei caratteri facendo uso di `xstrchr`.

4.4. Moduli `massmail.c` e `massmail.h`

Il modulo malevolo analizzato implementa un motore per la diffusione automatizzata di e-mail (*mass-mailing*) articolato in diverse fasi operative. L'inizializzazione del sistema fa capo alla funzione `massmail_init`, che provvede ad azzerare i puntatori della coda dei messaggi

`massmail_queue`, della cache dei server DNS `mm_dnscache` e del contatore dei thread attivi `mmshed_run_threads`. L'avvio dell'elaborazione in background viene gestito dalla funzione `massmail_main_th`, la quale esegue la routine principale `massmail_main` all'interno di un thread dedicato per poi terminarlo tramite la funzione `ExitThread`. Il ciclo principale del malware risiede nella funzione `massmail_main`, che controlla continuamente lo stato della connessione a Internet tramite `is_online`, sospende o riattiva la scansione dei moduli esterni invocando `scan_freeze`, ripulisce la coda eliminando gli elementi già elaborati e scaduti tramite `mmshed_rmold` (che ne libera la memoria mediante `HeapFree`), avvia la generazione sintetica di nuove e-mail con `mm_gen` in presenza di poche e-mail non elaborate e distribuisce il carico creando thread di spedizione tramite `CreateThread`.

La fase di validazione dei bersagli è gestita dalla funzione `email_filter`, che invoca `cut_email` per isolare e pulire l'indirizzo e-mail da spaziatura o caratteri spuri, richiama `email_check2` per verificare i vincoli di sintassi (la quale sfrutta a sua volta `email2parts` per separare nome utente e dominio e analizzare la percentuale di cifre numeriche), applica `email_filtldom` per scartare domini appartenenti a società di antivirus o enti governativi e utilizza `email_filtuser` per escludere account amministrativi o generici (impiegando anche `rot13` per identificare la stringa "spam"). Gli indirizzi considerati validi da `email_filter` vengono presi in carico da `massmail_addq`, che verifica l'assenza di duplicati nella coda `massmail_queue`, alloca la memoria della struttura con `HeapAlloc` e incrementa la priorità se il dominio appartiene all'ambito universitario `.edu`. Per incrementare autonomamente l'elenco dei bersagli, la funzione `mm_gen` sceglie casualmente un dominio da un elemento della coda `massmail_queue`, lo unisce a un nome estratto dall'array `gen_names` e riaggiunge l'indirizzo risultante invocando `massmail_addq`.

Per la risoluzione dei server di posta del destinatario, la funzione `mm_get_mx` consulta in prima battuta la cache locale tramite `mmdns_getcached` e, in caso di mancata corrispondenza, invoca `get_mx_list` per effettuare la query DNS, memorizzando successivamente il risultato con `mmdns_addcache` (la quale provvede anche a liberare eventuali record precedentemente salvati tramite `free_mx_list` e ad allocare nuova memoria con `HeapAlloc`). La composizione e l'invio del messaggio e-mail malevolo sono svolti dalla funzione `mmsender`, che ottiene le informazioni sui server MX invocando `mm_get_mx`, genera il corpo dell'e-mail tramite `msg_generate`, trasmette il messaggio mediante `smtp_send` e libera le risorse della memoria con `GlobalFree`. Infine, la funzione `mmsender_th` agisce da wrapper concorrente per l'invio, incrementando e decrementando atomicamente la variabile di conteggio thread `mmshed_run_threads`, aggiornando lo stato dell'elemento in coda, richiamando la funzione `mmsender` e chiudendo l'esecuzione del thread con `ExitThread`.

4.5. Moduli `msg.c` e `msg.h`

Il codice analizzato, composto dai moduli `msg.h` e `msg.c`, è incentrato sulla funzione principale `msg_generate`, la quale si occupa di costruire un messaggio di posta elettronica con allegato, potenzialmente malevolo. La funzione `msg_generate` inizialmente imposta l'indirizzo email del destinatario e poi invoca `select_from` per scegliere in modo pseudocasuale un indirizzo email del mittente, che può derivare da una coda di email preesistente o essere generato casualmente con un dominio codificato in ROT13. Successivamente, `select_exename` viene chiamata per definire il nome e l'estensione di un file eseguibile da allegare; questo può essere un nome generato casualmente o selezionato da un elenco predefinito di nomi e estensioni (come "document", "readme", "doc", "txt", "file", "data", "test", "message", "body" combinati con "pif", "scr", "exe", "cmd", "bat"), tutti codificati in ROT13. La funzione `select_subject` è poi invocata per determinare l'oggetto dell'email, scegliendo tra stringhe casuali o soggetti predefiniti e codificati in ROT13, applicando inoltre modifiche casuali alla capitalizzazione.

L'operazione successiva è gestita da `select_attach_file`, che prepara l'allegato decidendo se comprimerlo in un file ZIP. Se viene utilizzato un file ZIP, viene creato un file temporaneo e il modulo eseguibile del programma viene archiviato all'interno; in alcuni casi, viene applicata una "name trick" per camuffare il nome dell'allegato all'interno dello ZIP con estensioni ingannevoli come `.doc`, `.htm`, o `.txt` prima dell'estensione reale dell'eseguibile. La funzione verifica inoltre che la dimensione del file allegato rientri in un intervallo specifico. Dopo la preparazione dell'allegato, `msg_generate` alloca un buffer per l'intero messaggio email.

La costruzione effettiva del messaggio inizia con `write_headers`, che genera gli header dell'email, includendo i seguenti componenti:

- indici di *boundary MIME* casuale, indirizzi "From", "To" e "Subject", la data corrente, la versione MIME, il Content-Type come `multipart/mixed`
- indici di priorità "X-Priority: 3" e "X-MSMail-Priority: Normal", utilizzando stringhe codificate in ROT13.

Successivamente, `write_body` compone il corpo del messaggio. Inizia con un messaggio standard di tipo MIME, poi aggiunge una parte `text/plain` il cui contenuto testuale è generato da `write_msgtext`, che può produrre testo casuale o selezionare frasi predefinite relative a errori di consegna o codifiche.

Infine, `write_body` include una parte `application/octet-stream` per l'allegato, che viene codificato in base64 dalla funzione `msg_b64enc`. La funzione `msg_b64enc` legge il file allegato a blocchi, lo codifica in base64 utilizzando `base64_t2q` per ogni blocco di 3 byte, e scrive i dati codificati nel buffer del messaggio, garantendo che le linee non superino una certa lunghezza.

Infine, `msg_generate` libera eventuali file temporanei e restituisce un puntatore al buffer contenente il messaggio email generato, gestendo anche la liberazione delle risorse in caso di errori.

4.6. Modulo `p2p.c`

Il modulo gestisce la propagazione del malware attraverso la rete Peer-to-Peer di Kazaa. La funzione principale `p2p_spread` avvia il processo recuperando il percorso completo del proprio eseguibile corrente tramite l'API `GetModuleFileName`, inoltrando quindi tale percorso alla funzione `kazaa_spread` per eseguire l'infezione. All'interno di `kazaa_spread`, la funzione decodifica le stringhe necessarie all'individuazione del registro di sistema (come la chiave `"Software\\Kazaa\\Transfer"` e il valore `"DlDir0"`) tramite la funzione `rot13`. Successivamente, `kazaa_spread` azzerava il buffer del percorso con `ZeroMemory`, apre la chiave di registro tramite `RegOpenKeyEx`, estrae il percorso della cartella di download condivisa di Kazaa mediante `RegQueryValueEx` e rilascia la risorsa con `RegCloseKey`. Ottenuta la cartella, `kazaa_spread` ne verifica la formattazione con `lstrlen` e `lstrcat`, quindi genera un nome di file trappola selezionando casualmente un elemento dall'elenco cifrato `kazaa_names` tramite la funzione `xrand16()`, decodificandolo con `rot13` e concatenando a quest'ultimo una tra le estensioni eseguibili supportate (`.exe`, `.scr`, `.pif`, `.bat`). Infine, la funzione `kazaa_spread` invoca l'API `CopyFile` per duplicare l'eseguibile malevolo originale nella directory condivisa di Kazaa sotto il nuovo nome generato.

4.7. Moduli `scan.c` e `scan.h`

La funzione `scan_init` inizializza il modulo impostando a zero la variabile globale `scan_freezed`, mentre `scan_freeze` ne aggiorna lo stato per mettere in pausa o riprendere l'attività di scansione. L'esecuzione principale è gestita da `scan_main`, la quale invoca prima `scan_default_wab` per cercare la rubrica di sistema predefinita, abbassa la priorità del thread con `SetThreadPriority`, chiama `scan_ietemp` per esplorare la cache di Internet ed entra infine in un ciclo permanente che esegue periodicamente `scan_disks`. Nello specifico, `scan_default_wab` decodifica la chiave di registro del file WAB tramite `rot13`, estrae il percorso della rubrica con `RegOpenKeyEx` e `RegQueryValueEx`, rilascia l'handle con `RegCloseKey` e trasmette il percorso a `scan_wab`. La funzione `scan_wab` apre il file `.wab` con `CreateFile`, ne proietta il contenuto in memoria mediante `CreateFileMapping` e `MapViewOfFile`, estrae i singoli indirizzi di posta memorizzati per inviarli a `scan_out`, e libera infine la memoria e le risorse con `UnmapViewOfFile` e `CloseHandle`.

Per quanto riguarda la scansione del file system, `scan_ietemp` ricava la directory di Windows o il profilo utente tramite `GetWindowsDirectory` e `GetEnvironmentVariable`, ricompone i percorsi delle cartelle temporanee decodificandole con `rot13` e le passa a `scan_dir1`. Allo stesso modo, `scan_disks` individua il disco di sistema con `GetSystemDirectory`, lo analizza mediante `scan_dir1` e itera successivamente attraverso le lettere di unità da 'C' a 'Z', selezionando con `GetDriveType` i volumi fissi o RAM disk su cui richiamare `scan_dir1`. Durante l'ispezione delle cartelle, `scan_dir1` naviga nell'albero delle directory usando `FindFirstFile`, `FindNextFile` e `FindClose`, inoltrando ogni file a `scan_dir_file`. Quest'ultima controlla l'estensione e la dimensione del file, verifica se la scansione è in pausa e smista l'elaborazione verso `scan_textfile` o `scan_wab`.

Durante l'elaborazione dei documenti di testo, `scan_textfile` legge il file a blocchi con `CreateFile`, `ReadFile` e `CloseHandle`, per poi passare i dati a `scantext_textcvrt`, la quale normalizza i caratteri e converte le entità o offuscamenti HTML (sfruttando `xmcmapi`, `memcpy`, `html_replace` e `html_replace2`). Infine, la funzione `scantext_extract_ats` analizza il buffer convertito per individuare il carattere `@`, convalida la struttura degli indirizzi mediante la tabella `mail_chars` e li passa a `scan_out`, che provvederà a inserirli nella coda del motore di mailing di massa richiamando `massmail_addq`.

4.8. Moduli `sco.c` e `sco.h`

Il modulo malevolo implementa un attacco Denial of Service (DoS) HTTP distribuito verso un bersaglio prestabilito, coordinato principalmente dalla funzione `scodos_main`. Questa funzione avvia l'operazione decodificando la stringa del dominio target tramite la funzione `rot13`, per poi verificare ripetutamente lo stato della connessione Internet tramite `is_online` (mettendo in pausa l'esecuzione con `Sleep` in caso di assenza di rete) e risolvere l'indirizzo IP dell'obiettivo invocando `gethostbyname`. Una volta configurata la struttura dell'indirizzo di rete `sockaddr_in` con la porta HTTP impostata tramite `htons`, la funzione `scodos_main` crea 63 thread paralleli invocando ripetutamente `CreateThread` ed esegue direttamente l'istanza finale dell'attacco richiamando la routine `scodos_th`.

All'interno dell'esecuzione concorrente, la funzione `scodos_th` provvede a decodificare la richiesta HTTP GET da inviare mediante `rot13`, riduce la priorità del thread corrente invocando `SetThreadPriority` insieme a `GetCurrentThread` ed entra in un ciclo continuo di bombardamento. In questo ciclo, `scodos_th` richiede la creazione di una connessione verso l'obiettivo invocando la funzione ausiliaria `connect_tv`, trasmette la richiesta HTTP formattata tramite la funzione `send` (misurandone la lunghezza con `lstrlen`), mette in pausa il thread per 300 millisecondi con `Sleep` e rilascia la connessione mediante `closesocket`. Qualora la funzione `scodos_th` venga invocata senza parametri validi, termina immediatamente l'esecuzione del thread tramite `ExitThread`.

La gestione a basso livello del canale di comunicazione è affidata alla funzione `connect_tv`, che inizializza un socket TCP mediante la funzione `socket` e lo configura in modalità non bloccante tramite `ioctlsocket`. Successivamente, `connect_tv` avvia il tentativo di connessione con `connect`, gestendo gli errori di memoria temporanei (`WSAENOBUFFS`) tramite `Sleep` o monitorando lo stato di attesa (`WSAEWOULDBLOCK`) ottenuti da `WSAGetLastError`. Per verificare il completamento della connessione entro il timeout stabilito, `connect_tv` prepara i set di descrittori tramite

le macro `FD_ZERO` e `FD_SET`, monitora lo stato del socket invocando la funzione `select` e ne valida l'esito tramite `FD_ISSET`. Se la connessione va a buon fine, `connect_tv` ripristina la modalità bloccante del socket invocando `ioctlsocket` e restituisce il descrittore attivo; in caso contrario, provvede a chiudere il socket tramite `closesocket` e a restituire valore nullo.

4.9. Moduli `xdns.c` e `xdns.h`

Il modulo `xdns` implementa un sistema di risoluzione e gestione dei record di posta (MX) di un dominio necessario alle operazioni del malware. La funzione principale `get_mx_list` gestisce l'acquisizione della lista dei server di posta tentando prima l'interrogazione tramite le API di sistema con la funzione `getmx_dnsapi` e, in caso di fallimento, ripiegando sull'interrogazione diretta dei server DNS di rete tramite `getmx_mydns`. Nello specifico, `getmx_dnsapi` provvede al caricamento dinamico della libreria `dnsapi.dll` mediante `GetModuleHandle` o `LoadLibrary`, individua il punto di ingresso di `DnsQuery_A` con `GetProcAddress`, esegue la query DNS per il record MX ed elabora la struttura dei risultati allocando la memoria necessaria tramite la macro `mx_alloc` (la quale richiama `HeapAlloc`). Diversamente, la funzione `getmx_mydns` recupera la configurazione dei server DNS del sistema operativo caricando la libreria `iphlpapi.dll` ed invocando `GetNetworkParams`, per poi scorrere la lista degli indirizzi IP ottenuti (risolvendoli all'occorrenza con `gethostbyname`) ed inoltrare la richiesta a ciascun server tramite la funzione `my_get_mx_list`.

L'interrogazione diretta a basso livello fa capo alla funzione `my_get_mx_list`, che richiama la routine `my_get_mx_list2` fino a due volte gestendo eventuali ritardi con `Sleep`. La funzione `my_get_mx_list2` crea un socket UDP non bloccante con `socket`, costruisce ed invia il pacchetto di query DNS grezzo mediante `mx_make_query` (la quale genera un ID casuale tramite `GetTickCount`, converte il dominio nel formato a etichette tramite `mx_dns2qname` e trasmette il buffer con `sendto`), attende la risposta monitorando il socket con `select`, riceve i dati di ritorno con `recv` ed elabora la risposta invocando `mx_parse_rr`. Quest'ultima analizza i record della risposta ignorando la sezione di domanda con `mx_skipqn` e decodificando i nomi dei domini tramite `mx_decode_domain`, consentendo a `my_get_mx_list2` di costruire la catena di nodi `mxlist_t`, liberare la lista temporanea con `mx_free_rrlist` e chiudere il socket con `closesocket`. Infine, la deallocazione completa della memoria della lista dei server MX restituita viene eseguita dalla funzione `free_mx_list`, che attraversa tutti i nodi e libera le risorse associate richiamando `mx_free` (ovvero `HeapFree`).

4.10. Moduli `xsmtp.c` e `xsmtp.h`

Il modulo composto dai file `xsmtp.h` e `xsmtp.c` implementa un motore autonomo per l'invio diretto di messaggi e-mail tramite il protocollo SMTP, privo di interazione utente e progettato per la diffusione del malware tramite posta elettronica. Il componente analizza e formatta il contenuto delle e-mail, risolve i domini dei server di destinazione tramite query DNS o conversioni IP, apre socket di rete TCP, gestisce la trasmissione di comandi offuscati (quali `EHLO`, `HELO`, `MAIL FROM`, `RCPT TO`, `DATA` e `QUIT`) e gestisce meccanismi di fallback per reperire server SMTP alternativi provando nomi di dominio comuni o leggendo la configurazione degli account di posta dell'utente dal registro di sistema di Windows.

La funzione principale `smtp_send` avvia la procedura estraendo l'intestazione del destinatario mediante la funzione `mail_extracthdr`, scorre la lista dei server MX primari risolvendo ciascun hostname in un indirizzo IP tramite `resolve` e tenta la spedizione. Qualora i server primari falliscano, `smtp_send` genera dinamicamente varianti dell'hostname del destinatario (come `mail.%s` o `smtp.%s`) tramite la funzione `wsprintf`, ne ottiene l'indirizzo IP richiamando `resolve` e tenta l'invio con `smtp_send_server`, lasciando come risorsa finale l'invocazione di `xsmtp_try_isp`. La funzione `xsmtp_try_isp` si occupa di identificare il server SMTP prefigurato nel sistema deoffuscando la chiave di registro interessata tramite `rot13`, accedendo ed enumerando le sottochiavi con `RegOpenKeyEx`, `RegEnumKey` e `RegQueryValueEx`, liberando gli handle con `RegCloseKey`, traducendo il server trovato in IP con `resolve` ed eseguendo infine la spedizione con `smtp_send_server`.

L'interazione con il server di posta viene orchestrata dalla funzione `smtp_send_server`, la quale estrae i campi dei mittenti e destinatari tramite `mail_extracthdr`, crea il socket di trasmissione mediante `socket`, avvia la connessione al server remoto con `connect`, attende la pronta risposta tramite `wait_sockread` ed esegue lo scambio dei comandi di benvenuto ed identificazione decifrati al volo con `rot13` e spediti tramite `smtp_issue`. Successivamente, `smtp_send_server` inoltra il corpo dell'e-mail a blocchi gestendo i punti a inizio riga e la chiusura dati tramite la funzione `send`, per poi notificare l'uscita ed eliminare il socket richiamando `closesocket`. La trasmissione dei comandi SMTP individuali è affidata a `smtp_issue`, che compone le stringhe formattate invocando `wsprintf`, trasmette i dati al socket con `send`, legge la riga di risposta del server tramite `recvline` e ne analizza il codice di stato numerico.

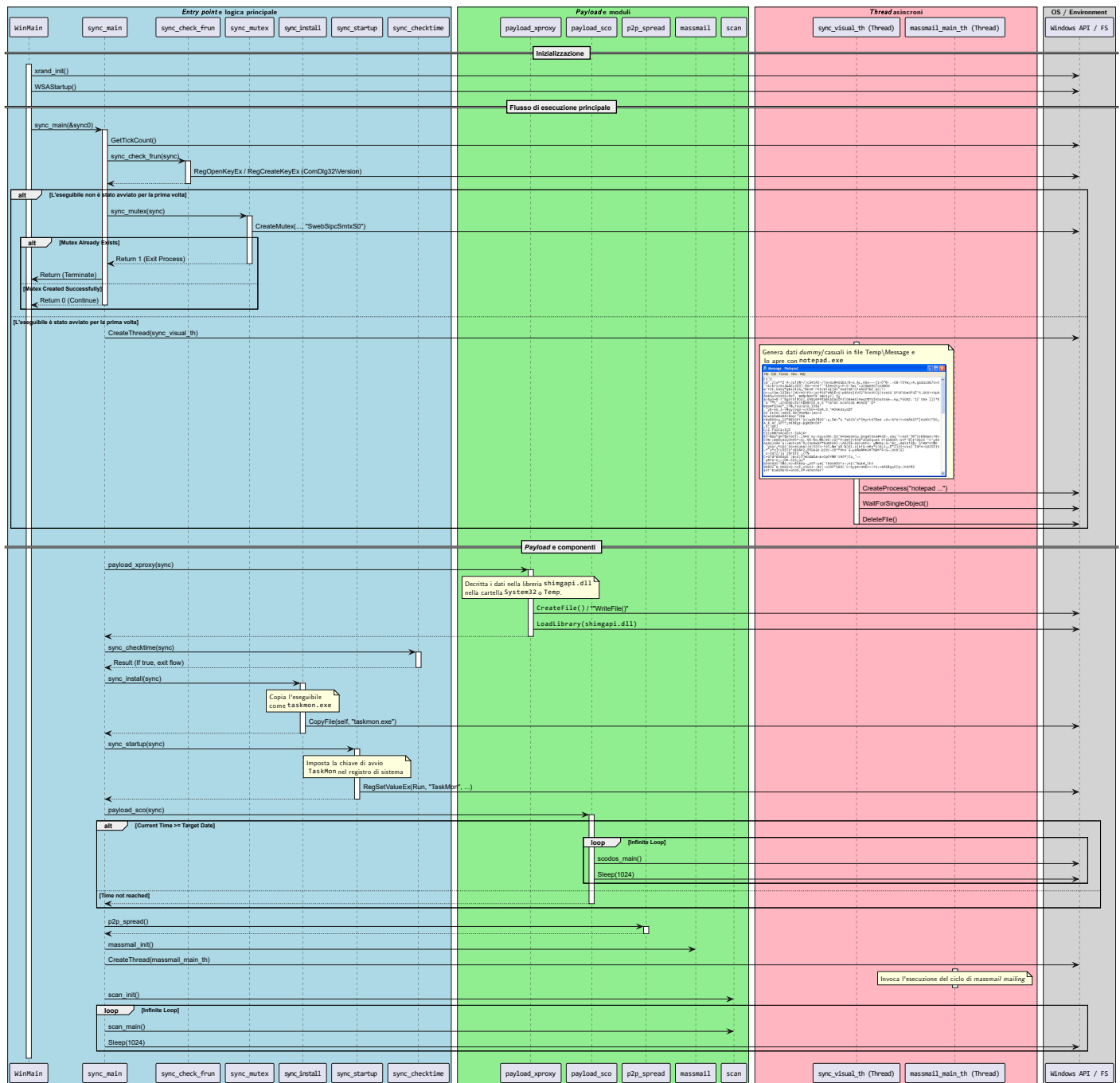
La ricezione a basso livello e la gestione del tempo di attesa della rete sono eseguite da `recvline`, che imposta il timeout e controlla la disponibilità del socket mediante `select` prima di leggere i dati byte per byte con `recv`. Allo stesso modo, `wait_sockread` alloca la struttura `timeval` e valuta lo stato del socket invocando `select`. La traduzione degli indirizzi di destinazione è gestita dalla funzione `resolve`, che prova a convertire l'IP in formato testo con `inet_addr` o risolve l'hostname di rete tramite la funzione `gethostbyname`. Infine, la funzione `mail_extracthdr` elabora il testo dell'e-mail per isolare e restituire le singole intestazioni cercate, normalizzando il testo minuscolo con `CharLower` e verificando la corrispondenza del nome dell'intestazione mediante la funzione `lstrcmpi`.

4.11. Moduli `zipstore.c` e `zipstore.h`

Il modulo offre funzionalità per l'archiviazione non compressa di file all'interno di un contenitore in formato ZIP. La funzione `zip_putcurtime` acquisisce l'orario corrente invocando la funzione di sistema `GetSystemTime`, valida e corregge i parametri di anno, mese e giorno se fuori scala e codifica i valori nei campi di data e ora conformi al formato bitfield MS-DOS. La funzione `crc32` esegue il calcolo dell'algoritmo CRC-32 su un buffer di byte sfruttando la tabella di ricerca `crc_table`. Per calcolare l'impronta dell'intero file sorgente, la funzione `zip_calc_crc32` invoca `SetFilePointer` per posizionarsi all'inizio del file, effettua chiamate cicliche a `ReadFile` per leggere blocchi da 1024 byte, invoca la funzione `crc32` per aggiornare progressivamente il checksum sui dati letti e ripristina la posizione del cursore iniziale richiamando nuovamente `SetFilePointer`.

La funzione `zip_store` si occupa di orchestrare l'intera creazione dell'archivio ZIP. Essa invoca `CreateFile` per aprire in lettura il file sorgente e creare il file di destinazione, gestendo le relative chiusure d'emergenza tramite `CloseHandle` in caso di fallimento. Successivamente azzerava le strutture di memoria per le intestazioni (`zip_header_t`, `zip_dir_t` e `zip_eod_t`) tramite `memset`, invoca `zip_putcurtime` per impostare la data e l'ora e richiama `zip_calc_crc32` per ottenere il checksum del file originale. In seguito, `zip_store` misura la dimensione del file sorgente invocando `GetFileSize` e calcola la lunghezza del nome del file da memorizzare tramite `lstrlen`, scrivendo l'intestazione locale e il nome del file nel file d'uscita mediante la funzione `WriteFile`. Per copiare i dati effettivi del file, `zip_store` riposiziona il lettore con `SetFilePointer`, legge i blocchi con `ReadFile` e li scrive direttamente nell'archivio mediante `WriteFile`. Infine, `zip_store` compila la directory centrale e il record di chiusura dell'archivio scrivendoli nel file con ulteriori chiamate a `WriteFile`, chiudendo definitivamente gli handler dei file mediante `CloseHandle`.

5. Diagramma di esecuzione del codice



6. Tattiche, tecniche e procedure del malware (TTP)

Sulla base dell'analisi dei moduli del codice sorgente, sono state identificate le seguenti tattiche, tecniche e procedure (TTP):

Evasione e offuscamento (defense evasion):

- E' rilevato un utilizzo sistematico dell'algoritmo ROT13 per cifrare chiavi di registro, nomi di file, comandi SMTP e parametri di rete, decifrando dinamicamente solo in memoria durante l'esecuzione.
- E' rilevata la decifratura della DLL proxy (`shimgapi.dll`) tramite routine XOR dinamica prima del salvataggio su disco e caricamento in memoria.
- E' rilevato l'impiego di estensioni doppie o ingannevoli (`.doc` , `.htm` , `.txt` seguite da estensioni eseguibili o spazi) ed estrazione all'interno di archivi ZIP non compressi generati al volo, a fini di camuffamento dell'allegato (*masquerading*)
- E' rilevata la creazione ed apertura di un file di testo fittizio contenente caratteri casuali mediante l'invocazione del software *Notepad*, o `notepad.exe` al primo avvio, al fine della simulazione di un errore di apertura documento e il *masquerading* dell'infezione in background

Esecuzione e persistenza (persistence & execution):

- Creazione e modifica di chiavi di avvio automatico, o *autorun*, nei rami `HKLM` e `HKCU` (`Software\Microsoft\Windows\CurrentVersion\Run`) con il valore `TaskMon`.
- Registrazione della libreria dinamica (DLL) proxy sotto la chiave `InprocServer32` per garanzia dell'esecuzione persistente e occultata come servizio di sistema.
- duplicazione del processo malevolo con diverso nome del file, tra cui nomi di eseguibili riconducibili agli eseguibili di sistema come `taskmon.exe`, nelle cartelle di sistema o temporanee.

Ricerca e collezione di dati (Discovery & Collection)

- Ricerca di indirizzi e-mail attraverso la scansione della rubrica di Windows, e i relativi file con estensione `.wab`, delle cartelle temporanee di Internet e della memoria di massa con lettere di volume dalla `C:` a `Z:`.
- *Deserializzazione* tramite funzioni di *parsing* di testi HTML/URL per l'estrazione di indirizzi tramite espressioni sulla sintassi (`@`) ed esclusione di domini appartenenti a società di sicurezza o account amministrativi, tra cui domini `"avp"`, `"syma"`, `"icrosof"` (si presume che la regola di esclusione sia definita per l'esclusione di domini dell'azienda *Microsoft*, ovvero che presentino la sottostringa `microsoft`), `"msn."`, `"hotmail"`, `"panda"`, `"sopho"`, `"borlan"`, `"inpris"`, `"example"`, `"mydomai"`, `"nodomai"`, `"ruslis"`, `".gov"`, `"gov."`, `".mil"` e `"foo."`.

Propagazione laterale e diffusione (Lateral movement & worming):

- Composizione sintetica di e-mail *MIME* multipart con mittenti/oggetti casuali e invio diretto via socket SMTP verso i server MX dei destinatari, risolti tramite query DNS native (`dnsapi.dll` / pacchetti UDP grezzi).
- Identificazione della cartella condivisa di Kazaa, denominata in `D1Dir0`, tramite registro e copia del malware con nomi trappola ed estensioni eseguibili (`.exe`, `.scr`, `.pif`, `.bat`), al fine della diffusione del vettore tramite il Diffusione protocollo `_ peer-to-peer _` (P2P) del software di condivisione file *Kazaa*.

Comando e controllo (C2) e backdoor (Command and Control & Backdoor):

- Creazione di un server *proxy* di tipo *SOCKS4 multithread* in grado di instradare traffico di rete remoto ed intercettare comandi speciali per la ricezione ed esecuzione nascosta di ulteriori payload binari.
- Creazione di un mutex di sistema (`CreateMutex`) per evitare infezioni sovrapposte sullo stesso host.

Impatto e attacco (Impact):

- Attivazione temporizzata di un attacco HTTP GET flooding verso un obiettivo predefinito, eseguito in modo concorrente tramite 63 thread paralleli, al fine di eseguire un attacco *Denial of Service* distribuito (DDoS)